Evaluating Cryptographic API Misuse Detectors for Go

Vivi Andersson
vivia@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Martin Monperrus
monperrus@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Abstract

Cryptographic API misuse represents a critical vulnerability class that undermines the security foundations of modern software. Yet, it remains largely unexplored in Go despite its dominance in security-critical infrastructure. This paper presents the first comprehensive study of cryptographic API misuse detection in Go, identifying and analyzing 4 state-of-the-art tools (CodeQL, Gopher, Gosec, and Snyk Code) and establishing a consolidated taxonomy of 14 relevant misuse classes. Through an experimental evaluation of 328 security-critical open-source Go projects, we discovered 7,473 cryptographic API misuses, providing insights into the prevalence and distribution of these vulnerabilities. Our systematic comparison reveals significant variations in misuse coverage, with immediate practical implications for security engineers and long-term implications for research in this domain.

CCS Concepts

• **Security and privacy → Software security engineering**; • **Software and its engineering → Software defect analysis**.

Keywords

Cryptographic API misuse,Security tool comparison,Go

ACM Reference Format:
Vivi Andersson and Martin Monperrus. 2026. Evaluating Cryptographic API Misuse Detectors for Go. In *4th International Workshop on Software Vulnerability Management (SVM '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3786165.3788440

1 Introduction

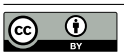
The misuse of cryptographic APIs is a critical class of code vulnerabilities [22]. Cryptographic API misuses are caused by the essential complexity of cryptographic protocols and the specialized knowledge required to use them correctly [2, 13, 25]. Yet, correct usage of these cryptographic APIs is required to maintain the security these protocols promise. When cryptographic APIs are wrongly used, the major problem is that developers have a false sense of cryptographic security, founded on mathematical truth, while the actual software is weak and insecure. For example, skipping certificate validation in TLS makes applications susceptible to man-in-the-middle attacks through nation-state firewalls [11].

Cryptographic misuse detection has been extensively studied in the Java ecosystem [7, 18, 23, 26, 28, 33, 34]. Yet, Java is not the stack used for most security-critical modern software infrastructure. Today, Go, aka Golang, is a major stack for infrastructure software[1]. It powers certificate authorities (e.g., Let's Encrypt's Boulder), service meshes (e.g., Traefik), container orchestration platforms (e.g., Kubernetes), and infrastructure-as-code tools (e.g., Terraform). Despite Go's inevitability in security-critical systems, cryptographic misuse detection remains understudied. To the best of our knowledge, no prior work has systematically analyzed cryptographic misuse detection for Go. We address this gap with the first qualitative and quantitative analysis.

Go has a comprehensive cryptographic API suite. For example, the Go library `crypto/tls` provides support for the protocol TLS. As in other ecosystems, Go developers remain prone to insecure code due to cryptographic API misuse [19, 36]. Compared to Java, Go's smaller standard library and more opinionated defaults can reduce certain algorithm-selection errors, shifting failure modes toward configuration and usage mistakes. Two recent, high-profile CVEs highlight these risks: CVE-2024-45337 in `golang.org/x/ssh`, where flawed assumptions about public-key authentication enabled authentication bypasses [31], and CVE-2025-66491 in Traefik, where `Verify=On` inverted verification semantics and disabled TLS certificate checks [24].

We define a *cryptographic API misuse* as the incorrect usage of cryptographic functions through an API, which introduces security vulnerabilities and compromises the theoretical guarantees of the cryptographic functions used. From there, we study the problem of cryptographic API misuse in Go as follows. First, we conduct a survey of tools for detecting cryptographic API misuse in Go. Next, we analyze those tools (incl. the corresponding paper and documentation) in order to create a consolidated taxonomy of cryptographic API misuse for Go. Second, we perform a quantitative experimental campaign. We collect security-relevant open-source projects in Go, run all the tools over them, and collect the detected API misuses. This enables us to study prevalence, tool consensus, and performance of misuse detection tools in Go.

Results. We study four state-of-the-art tools spanning academic, open-source, and commercial origins: CodeQL, Gopher, Gosec, and Snyk Code. We consolidate 14 cryptographic misuse classes that (i) are relevant in Go and (ii) are supported by at least one tool; several have appeared in recent CVEs. Running all four tools on 328 open-source projects yields 7,473 detected API misuses, enabling a systematic comparison across misuse classes. These findings help practitioners (e.g., Kubernetes security engineers) choose and interpret tools, acknowledging, for instance, that not all provide character-level precision, and highlight research needs, notably the formalization and enforcement of API-level security



SVM '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2397-1/2026/04
https://doi.org/10.1145/3786165.3788440

[1] https://go.dev/wiki/GoUsers

invariants. We also release a replication package to facilitate follow-up work.[2]

Contributions.

- The first systematic analysis of the crypto-misuse problem space in the Go infrastructure domain.
- A large-scale experiment collecting 7,473 misuses across 328 notable open-source projects using four state-of-the-art detection tools.

2 Experimental Methodology

2.1 Research Questions

The objective of this research is to evaluate the state-of-the-art in detecting cryptographic API misuse in Go. We aim to identify and analyze the strengths and weaknesses of existing approaches, structuring the evaluation around two research questions:

RQ1. What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go? In this RQ, we aim to 1) identify the existing tools applicable to go, 2) which types of misuses they detect, and 3) by which methods. We first identify relevant tools through a systematic literature search (subsection 2.2), then analyze their technical design and detection capabilities. For academic tools, we extract information primarily from the corresponding paper. For industrial tools, we survey the official documentation. We classify each tool based on its analysis approach and categorize the types of cryptographic vulnerabilities each tool detects.

RQ2. To what extent are crypto misuse detection tools for Go consistent with each other? In this RQ, we aim to quantitatively compare the concrete performance of the considered tools when applied to real-world Go projects. We start by assembling a representative dataset of Go modules that depend on cryptographic libraries for application security (subsection 2.3). Each tool is then run over this dataset, and its detections are matched and compared.

2.2 Considered Tools

We conduct a systematic search to identify tools for detecting cryptographic API misuse in Go. We start by searching Google Scholar using the query `'crypto* API misuse detection AND tool'`. We manually review each paper and confirm that it proposes a tool with Go support. Then, we conducted a broader web search targeting industry and community-developed tools. Since not all of them may use the terminology of "misuse", we focus the search query on the keyword `crypto vulnerability`.

We curate a systematic list of tools that meet the following inclusion criteria: (1) Support for Go, (2) Capability to detect cryptographic API misuse, (3) Available documentation, (4) Active development (e.g., a commit in the last 6 months).

Our search yields five tools, of which four meet our criteria. From academia, we identify *Gopher* [36], a specialized Go API misuse detection tool. One academic tool *CryptoGo* [19] is no longer available and is therefore excluded. From the industry, we find three actively maintained tools with Go support. *CodeQL* [1] is GitHub's Datalog-based framework with built-in Go analysis and rules for detecting cryptographic API issues. *Gosec* [9] is a community-developed security checker for Go, and *Snyk Code* [29] is a proprietary, free-of-charge, static application testing tool from the company Snyk. To the best of our knowledge, this is the most up-to-date and comprehensive list of tools to detect cryptographic API misuse in Go.

2.3 Open-source Projects Dataset

We base our quantitative evaluations on a dataset of real-world Go projects from Chen et al. [6]. The original dataset includes 920 unique, highly-starred repositories collected via crawling GitHub and systematic filtering (removing duplicates, archived, inactive, and educational repositories). We further refine this dataset as follows.

Repository Health. We remove archived or unavailable modules and retain only well-maintained projects with commits in the past year and stable releases (indicated by non-pre-release Git tags).

Cryptographic Relevance. We exclude projects that do not import any Go standard `crypto` packages. Following prior work [7, 33, 35], we manually assess each project's cryptographic security relevance, classifying project types and retaining only those depending on cryptography for core functionality. For example, *Gops*[3] is excluded since cryptography is not central to its purpose.

Descriptive Statistics. The final dataset comprises 328 security-relevant projects, ranging from 7 to over 8 million lines of code (median: 58,761) (Figure 1). These actively maintained projects have a median of 9,096 stars and 167 contributors, reflecting strong community adoption.

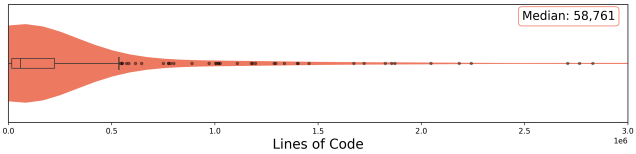


Figure 1: Lines of code per repository

2.4 Protocol for RQ1

We conduct a systematic qualitative comparison of the 4 tools identified in subsection 2.2: CodeQL, Gopher, Gosec, and Snyk Code. We compare the detection coverage across tools and their technical implementation.

Taxonomy Rules. To enable systematic comparison across tools with divergent coverage and classification approaches, we establish a unified taxonomy of cryptographic misuse issues. Each rule in our taxonomy represents a distinct type of cryptographic API misuse that can be detected through static analysis. We begin by collecting all distinct detection rules from each tool, preserving their original granularity and categorization. This forms a baseline of available issues detected across all evaluated tools. We then merge overlapping rules into common categories. For example, CodeQL uses a single rule "CWE-327" for detecting TLS issues, while Gopher represents these as 2 distinct rules. We merge these

[2] https://github.com/ASSERT-KTH/crypto-api-misuse-detectors-go

[3] https://github.com/google/gops

into a unified TLS security category. This consolidation creates a comparable taxonomy across all evaluated tools. We map each issue to security advisories where available.

We retain each rule's original severity classification (High, Medium, Low) as reported by the source. For each tool, we assess which of the rules is supported. We do so by surveying the accompanying paper and official documentation.

2.5 Protocol for RQ2

We evaluate the tools' practical detection capabilities by running each tool on our dataset and comparing their detections.

2.5.1 Tool Configuration and Execution. We select tools that are available and executable, using only their built-in rules. We exclude reports in test files when possible, but do not apply further configuration for filtering alerts. Each tool is executed on every applicable project in our dataset using the mapped rules relevant to that tool.

2.5.2 Cross-tool Detection Matching. We map tool-specific rules to our unified taxonomy (subsection 3.1), then match detections by location and rule. For per-rule agreement, we match by (project, file, line, rule), counting each tool once per combination. This captures line-level consensus on specific vulnerability types. For overall agreement, we match by (project, file, line) to measure whether tools agree that a line contains *any* vulnerability. Column positions are omitted as they are inconsistently reported across tools.

2.5.3 Evaluation Metrics. Without ground truth for tool detections, we rely on *manual analysis* of detections. We also report median wall-clock *execution time* per tool:

$$\text{Execution Time}_t = \text{median}_{p \in P} \quad (\text{Time}(t, p))$$

where $\text{Time}(t, p)$ represents the total execution time for tool t on project $p \in P$ when analyzing all supported rules.

2.5.4 Experimental Setup. We conducted our experiments on a server running Ubuntu 22.04, equipped with a 36-core Intel Core i9-10980XE CPU at 3.00 GHz and 125 GB of RAM. Each tool-project pair runs in a Docker container with 2 CPUs and 9 GB of memory. We use GNU Parallel[4] for scheduling.

3 Qualitative Analysis

3.1 Taxonomy of Cryptographic API Misuses

Our final taxonomy includes 14 distinct misuse issues grouped into six domains. Table 1a) summarizes these issues, their severity, and related advisories. Advisories prefixed *GO* refer to entries in the Go vulnerability database[5] . Each category reflects real-world patterns observed in production, supported by their implementation in existing tools and associated security advisories.

3.1.1 Cryptographic Primitives. This category targets the usage of cryptographic algorithms that are considered fundamentally insecure or deprecated.

Insecure Algorithms. (#01) covers the use of broken cryptographic primitives. For instance, MD5 has been considered insecure since 2004 due to practical collision attacks [15]. Its continued use in security-critical contexts leads to vulnerabilities. For example, the Go web framework *Beego* used MD5 to generate cache keys (CVE-2024-55885), risking data integrity. Collisions could let attackers overwrite legitimate cache entries, serving malicious responses or enabling unauthorized access.

Insecure Random Number Generation. (#02) covers the use of non-cryptographic randomness in security contexts. Go's `math/rand` package is predictable and thus insecure [10]. For example, the *Caddy* web server's `caddy-security` plugin (CVE-2024-21495) used `math/rand` to generate OAuth nonces, allowing attackers to predict values and perform replay attacks with forged requests.

Deprecated Libraries. (#03) relates to the usage of code in the standard library that is considered inefficient, unmaintained, or error-prone. As an example, the `crypto/elliptic`[6] package has been deprecated, superseded by `crypto/ecdh`, which provides safer defaults.

3.1.2 Key Management. This category collects issues arising from improper generation, management, and usage of cryptographic keys and parameters. Unlike algorithmic weaknesses, where the primitive itself is flawed, these vulnerabilities arise from poor key practices that undermine the otherwise secure cryptographic systems.

Constant/Predictable Keys. (#04) concerns the use of static, hard-coded, or guessable cryptographic keys. In *Kiali* (CVE-2020-1764), a default JWT signing key allowed attackers to forge tokens and gain unauthorized administrative access to the API and logs.

Short Key Length. (#05) addresses cryptographic keys shorter than current security standards. HMAC keys should be at least 128 bits to resist brute-force attacks [30]. For example, *DataHub* (CVE-2023-47640) used an 80-bit SHA-1 HMAC for cookie signing, allowing attackers to brute-force the key and gain elevated privileges.

Static or Predictable IVs. (#06) concerns the need for unique, unpredictable initialization vectors (IVs). Static or predictable IVs in block ciphers expose ciphertexts to dictionary attacks [21]. For instance, *NetBird* (CVE-2024-41260) used a hardcoded IV in CBC encryption, causing deterministic outputs that let attackers detect repeated plaintexts or build ciphertext dictionaries.

3.1.3 Password-based Key Derivation Functions. This category covers issues in the cryptographic use of passwords, specifically password-based key derivation functions (PBKDFs). Algorithms such as Argon2 [5] and bcrypt [27] convert passwords into cryptographic keys through computationally intensive operations, making brute-force guessing significantly harder.

Short Salt Length. (#07) in PBKDFs increases the risk of hash collisions across users, making lookup table attacks more feasible. For a small salt space, attackers can precompute hash values for common passwords and reuse them [27].

[4] https://www.gnu.org/software/parallel/
[5] https://pkg.go.dev/vuln/
[6] https://pkg.go.dev/crypto/elliptic

Table 1: Detection of cryptographic misuses by tools for Golang, per their documentation.

(a) Taxonomy of Cryptographic API Misuses in Go. **(b)** Detection support across tools.

Category	ID	Issue	Sev.	Advisory	CodeQL	Gopher	Gosec	Snyk
Cryptographic Primitives	01	Insecure algorithms	High	CVE-2024-55885		✓	✓	✓
	02	Crypto insecure PRNG	Med	CVE-2024-21495	✓	✓	✓	✓
	03	Deprecated Go function	Low	–		✓		
Key Management	04	Constant/predictable key	High	CVE-2020-1764		✓		
	05	Short key length	Low	CVE-2023-47640†	✓	✓	✓	✓
	06	Static or predictable IV	Med	CVE-2024-41260		✓	✓	
Password-based KDF	07	Short salt length	Low	–		✓		
	08	Predictable salt	Low	–		✓		
	09	Low hash iterations	Low	CVE-2023-46233†		✓		
Transport Security	10	HTTP protocol	High	CVE-2024-1968†		✓		
	11	TLS/SSL Issues	High	CVE-2024-23656	✓	✓	✓	✓
Secure Shell	12	Insecure SSH suite	High	CVE-2021-32026		✓		
	13	No host key validation	High	CVE-2024-41264	✓	✓	✓	
Token Auth.	14	No JWT verification	High	CVE-2024-51744	✓			

Note. † Non-Go advisories; – no associated advisory.

Predictable Salts. (#08) addresses vulnerabilities from low-entropy or static salt generation. Hardcoded or predictable salts make hashes guessable via precomputed lookups. For instance, the *Duplicacy* backup tool[7] used a hardcoded salt, undermining salting and enabling lookup-table attacks.

Low Hash Iterations. (#09) in PBKDFs reduces computational cost for attackers, enabling faster brute-force attacks. For instance, a low bcrypt `cost` parameter[8] weakens password protection. A notable real-world case is `crypto-js`, which defaulted to a single PBKDF2 iteration (CVE-2023-46233), leaving users exposed to significantly reduced security.

3.1.4 Transport Security. This category concerns the secure transmission of data over networks and comprises two rules.

HTTP Protocol. (#10) concerns transmitting sensitive data over unencrypted HTTP, exposing it to eavesdropping and man-in-the-middle attacks. This occurs when applications send credentials or personal data without HTTPS. For example, CVE-2024-1968 affected the Scrapy framework, which failed to remove Authorization headers during HTTPS-to-HTTP redirects, leaking authentication tokens in plaintext.

SSL/TLS Issues. (#11) covers three classes of vulnerabilities in SSL/TLS configuration. First, outdated protocol versions (SSL 3.0, TLS 1.0, 1.1) contain design flaws [16], enabling attacks like POODLE [8]; for example, CVE-2024-23656 in Dex exposed deprecated TLS versions. Second, insecure cipher suites rely on weak algorithms such as RC4 or lack forward secrecy. Third, skipping certificate verification allows server impersonation via man-in-the-middle attacks.

3.1.5 Secure shell (SSH). issues concerns the configuration and usage of the Go `crypto/ssh`[9] package.

[7] https://github.com/gilbertchen/duplicacy/issues/137
[8] https://pkg.go.dev/golang.org/x/crypto/bcrypt
[9] https://pkg.go.dev/golang.org/x/crypto/ssh

Table 2: Tool properties for misuse detection.

Tool	Available	Open Source	Rule Selection	Column Prec.	Severity	Confidence
CodeQL	✓	✓	✓	✓	✓	✗
Gopher	✓	✗	✓	✗	✗	✗
Gosec	✓	✓	✓	✓	✓	✓
Snyk	✓	✗	✓	✓	✓	✗

Insecure SSH Suite. (#12) covers configurations using outdated ciphers (e.g., CBC mode), which enable plaintext recovery via padding oracle attacks [3]. Although Go's SSH package disables these by default, users can re-enable them, as seen in *NATS* (CVE-2021-32026), where insecure overrides weakened security.

No Host Key Verification. (#13) arises when SSH key exchange skips host verification, enabling man-in-the-middle attacks. In Go, this typically results from using `ssh.InsecureIgnoreHostKey`. *Casdoor* (CVE-2024-41264) exhibited this flaw, allowing attackers to impersonate trusted servers and intercept sensitive data.

3.1.6 Token-based Authentication. This category includes one misuse related to JSON Web Tokens (JWTs), widely used for web authentication [17].

No JWT Verification. (#14) refers to missing signature checks, allowing attackers to forge tokens and access protected resources [17]. In `golang-jwt/jwt`[10] (CVE-2024-51744), unclear error-handling documentation led developers to accept invalid tokens.

3.2 Tool Characteristics

We compare key characteristics distinguishing the tools in our study. Table 2 summarizes their availability, configurability, and analytical capabilities.

Availability & Open-source. All four tools are publicly available. CodeQL and Gosec are fully open-source[11], allowing modification

[10] https://github.com/golang-jwt/jwt
[11] https://opensource.org/osd

Listing 1: Simplified Rule 02 (*CWE-338*) detection in CodeQL.

```
class InsecureRandomSource extends Source, DataFlow::CallNode {
  InsecureRandomSource() { getTarget().getPackage().getPath() = "math/rand"
        }}

class CryptographicSink extends Sink {
  CryptographicSink() {
    exists(Function fn, string pkg, string name |
      pkg.regexpMatch("crypto/.*") and not (pkg="crypto/rand" and
            name="Read") and
      exists(DataFlow::CallNode c | c.getTarget()=fn and
            this=c.getAnArgument()))
  } }
```

and redistribution. In contrast, Snyk Code and Gopher are closed-source: Snyk Code as a commercial product and Gopher as a research binary-only.

Custom Rules. All tools support adding new detection rules, but with varying flexibility. CodeQL and Gosec offer fine-grained control, allowing inclusion/exclusion of specific rules for targeted analysis. Gopher and Snyk Code lack this suppression feature, forcing users to apply their full rule sets.

Detection Output Characteristics. The tools differ in how precisely they report findings (granularity, severity, confidence). Most tools (CodeQL, Gosec, Snyk Code) report at column-level precision, while Gopher reports only by line. All except Gopher assign severity levels, and only Gosec includes confidence ratings, helping users prioritize results.

Finding 1 The tools vary in availability, configurability, and detection precision. An ideal tool offers rule selection, column-level accuracy, and severity and confidence ratings. Only *Gosec* meets all these criteria.

3.3 Detection Capabilities

We analyze each tool's coverage of cryptographic API misuse issues and their detection methods, assessing how comprehensively they address our taxonomy (Table 1b).

CodeQL. detects five misuse rules, emphasizing high-severity issues (three of five). It uniquely identifies unverified JWTs in `golang-jwt/jwt` (#14). CodeQL operates in two stages: first, it converts the codebase into a fact database capturing syntactic and semantic relationships [1]; second, QL queries analyze this data, often through data-flow reasoning. For instance, Rule #02 (shown in Listing 1) models insecure randomness as a taint-analysis problem, tracking flows from `math/rand` to `crypto` APIs while excluding benign cases.

Gopher. covers 13 of the 14 rules (excluding JWT authentication). It targets multiple Go standard library APIs. Gopher applies taint tracking on the SSA representation [36] and supports dynamically updatable rules. For example, when encountering an indirect reference to a cryptographic primitive (e.g., `rsa.GenerateKey`), it can add new rules to detect misuses that do not directly invoke known APIs.

Gosec. covers six rules across four categories. Like Gopher, it uses SSA analysis for certain checks (e.g., short keys, static IVs) [9], while others rely on simpler AST inspection. For instance, insecure randomness (#02) is detected by matching function calls against a blacklist.

Snyk Code. supports four rules overlapping with including insecure algorithms (#01) and SSL/TSL misconfigurations (#11). It applies static analysis enhanced by an "AI engine" [29], though implementation details remain proprietary.

Finding 2 Coverage ranges from 4 (*Snyk Code*) to 13 (*Gopher*) of 14 patterns. Primary detection mode is taint tracking over intermediate representations.

4 Quantitative Analysis

We examine how effectively the 4 considered tools that are available (CodeQL, Gopher, Gosec, Snyk Code) detect cryptographic API issues across our dataset of 328 open-source projects.

4.1 Detection Prevalence

4.1.1 Reliability and Coverage. Table 3 shows variability in tool reliability and coverage. Snyk Code and Gosec analyze all projects, while Gopher fails on 75 (22.9%) and CodeQL on 27 (8.2%).

Detection coverage varies widely: Snyk Code flags 92.7% of all projects, Gosec 84.5%, Gopher 50.0%, and CodeQL 28.4%. Most projects are flagged by at least one tool.

4.1.2 Detection Patterns. Table 4 summarizes detection magnitude per tool and misuse category across the dataset. For instance, CodeQL reports 14 instances of insecure PRNG usage. Missing support for a rule is indicated by a "–".

Detection varies widely, even within the same category (e.g., #11). Such discrepancies stem from differing definitions of what constitutes a misuse. For example, in rule #02 (insecure PRNGs), Gopher, though claiming support, reports none, while Gosec flags 2,517, CodeQL 14, and Snyk Code 17. While it is known that assumptions may differ, having such differences makes it very hard for researchers and practitioners to compare performance. Gopher shows the broadest coverage, uniquely detecting issues for rules #03, #04, #07–#10, and #12. However, rules #03 (deprecation) and #10 (HTTP) yield excessive warnings, suggesting overly aggressive heuristics.

Snyk Code detects four rule types, particularly flagging insecure algorithms (#01) 1,259 times, mostly involving MD5 and SHA-1 usage. Gosec also reports these, but only at import statements, producing fewer findings.

Manual analysis shows that the considered tools do not distinguish whether these usages are security relevant or not. For

Table 3: Tool execution outcomes across the dataset.

	CodeQL	Gopher	Gosec	Snyk
Projects analyzed (%)	91.8	77.1	100	100
With findings (% of analyzed)	30.9	64.8	84.5	92.7
With findings (% of dataset)	28.4	50.0	84.5	92.7

Table 4: Detection counts per rule across analysis tools.

ID	Description	CodeQL	Gopher	Gosec	Snyk	Total
01	Insecure algorithms	–	498	584	1259	2341
02	Crypto insecure PRNG	14	–	2517	17	2548
03	Deprecated Go function	–	99	–	–	99
04	Constant/predictable key	–	44	–	–	44
05	Short key length	11	12	7	5	35
06	Static/predictable IV	–	5	11	–	16
07	Short salt length	–	18	–	–	18
08	Predictable salt	–	13	–	–	13
09	Low hash iterations	–	59	–	–	59
10	HTTP protocol	–	197	–	–	197
11	TLS/SSL Issues	210	300	1049	474	2033
12	Insecure SSH suite	–	5	–	–	5
13	No host key validation	15	16	28	–	59
14	No JWT verification	6	–	–	–	6
	All	256	1266	4196	1755	7473

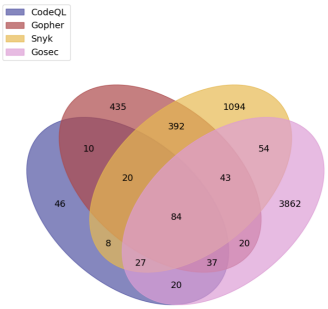


Figure 2: Detection agreement across all rules shows that many detections are tool-unique.

instance, Gopher flags all occurrences of `chacha20.NewUnauthenticatedCipher`, even when the output is subsequently authenticated (e.g., with Ed25519).

> **Finding 3** Snyk Code and Gosec run reliably on all projects; Gopher fails often yet detects the widest range, sometimes excessively. Large detection discrepancies reveal inconsistent assumptions complicating comparison.

4.2 Tool Consensus

We measure agreement at the line level, and for rule-specific analysis, require the same rule at the same line. High overlap suggests stronger confidence, while tool-specific detections may reflect unique capabilities or false positives; see subsubsection 2.5.2.

4.2.1 Aggregate Agreement. Across 14 misuse types, we identify 6,152 unique detections. Gosec accounts for most (67.4%), followed by Snyk Code (28.0%), Gopher (16.9%), and CodeQL (4.1%) (Figure 2). Most findings are tool-specific: Gosec reports 92.6% unique locations, Snyk Code 63.5%, suggesting complementary techniques but also false positive risk. Conversely, 81.7% of CodeQL alerts overlap with others. Only 1.3% of detections are shared by all tools, underscoring fragmented, immature detection in Go.

4.2.2 Rule-Specific Agreement. We examine three representative rules with the highest tool coverage (#05, #11, #13) to reveal finer-grained consensus patterns (Figure 3).

Short Key Lengths. (#05) is a covered by all four tools. Figure 3a is the exhaustive analysis of the consensus over all flagged alarms. Gopher reports 11 unique detections missed by others, while CodeQL, Gosec, and Snyk Code agree on 5 locations not flagged by Gopher. During manual analysis we observe that 4 of 5 issues flagged by all 3 tools, except Gopher, are security-irrelevant; they occur in mock and example code and are unlikely to end up in production code. There is one relevant detection that Gopher misses concerning an RSA key length of 1024, representing a false negative.

Gopher 's 11 unique detections reflect a broader interpretation of the rule. Six occur in one project, where it flags unkeyed hashes such as `blake2b.New512(nil)`, which are intended for hashing rather than MACs[12]. Since static analysis cannot infer context, Gopher conservatively flags these cases. In another project, Gopher identifies low-entropy HMAC usage through a wrapped function call. Although this instance is used for file fingerprinting and not security-relevant, it demonstrates how dynamic rule updates allow Gopher to detect issues missed by other tools. Overall, for this rule, Gopher provides the most advanced detection. With improved robustness (Table 3), it could become a highly effective tool for identifying insecure code patterns.

SSL/TLS Issues. (#11) yield 1,391 line-level detections, with overlaps shown in Figure 3b. All tools agree on 80 detections, while Gosec and Snyk Code dominate unique findings with 795 and 209, respectively. We randomly sampled five detections from each set.

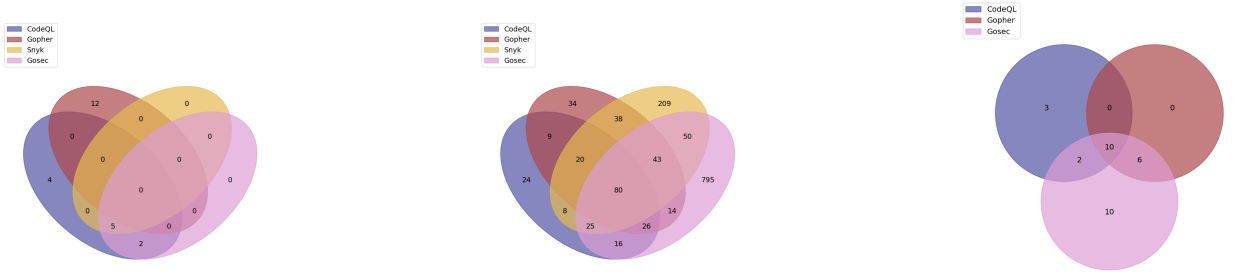
Gosec mainly flags TLS version settings and disabled certificate validation via `InsecureSkipVerify`. Three of five samples are false positives where TLS is properly configured (≥ 1.2), or validation is not actually disabled, suggesting limited sensitivity to configuration values. The remaining two cases are insecure only if developers explicitly enable risky options.

Snyk Code shows no false positives among its five samples: three correctly identify intentional insecure configurations, and two point to potentially security-relevant issues, indicating stronger contextual awareness than Gosec. For the five detections shared by all tools, all concern `InsecureSkipVerify` and are true positives: four clearly disable verification, and one reimplements it through a custom method.

SSH Host Key Validation. (#13) issues are reported by CodeQL, Gopher, and Gosec. These detections focus on how the SSH `HostKeyCallback` is implemented. Across tools, there is agreement on 10 alerts, with CodeQL identifying 3 unique cases and Gosec 10. We sampled five alerts from each tool's unique findings.

Gosec behaves similarly to rule #05: it detects true positives of the well-known `InsecureIgnoreHostKey` pattern (1) but also includes a testing file (1) and misses contextual cues such as custom callbacks (1) or explicitly benign configurations (2).

[12]https://pkg.go.dev/golang.org/x/crypto/blake2b



(a) Rule 05 (short key length) **(b) Rule 11 (SSL/TSL issues)** **(c) Rule 13 (No key validation)**

Figure 3: Tool detection overlaps for 3 representative rules. Rule 05 is extreme in the lack of agreement. Rule 11 shows that many detections can be separately confirmed by 2 tools. Rule 13 illustrates rare detections, appropriate for manual analysis.

Samples where all three tools overlap (five alerts) show higher precision, with all being true positives. One of these is intentionally configurable for contexts where no known hosts are available.

CodeQL stands out by detecting issues beyond the basic `Insecure IgnoreHostKey` pattern. It also identifies cases where custom SSH callbacks disable host verification by returning `nil`, demonstrating its capacity to analyze higher-order functions and complex data flows.

Finding 4 Detection profiles vary by type: Gopher offers broad coverage, Snyk Code context-aware precision, CodeQL unique SSH detection (#13), while Gosec produces most false positives. Cross-tool agreement correlates with true positives, suggesting ensemble approaches.

4.3 Execution Time

Table 5 reports median execution time per project, which matters for feasibility in CI and for scaling to large corpora.CodeQL has the highest overhead due to database creation (144s), for a median total of 219s. Snyk Code is fastest at 16s even when running all available rules. Gosec follows at 29s, while Gopher is slower at 63s, reflecting its broader rule coverage.

Finding 5 CodeQL is the most time-consuming tool, while Snyk Code runs 13× faster on median.

Table 5: Median time for setup and analysis per project.

	CodeQL	Gopher	Gosec	Snyk*
Setup (s)	144	–	–	–
Analysis (s)	70	63	29	16

Note. *For all available Snyk rules, including irrelevant ones.

5 Threats to Validity

External Validity. Our 2021 dataset may not capture newer Go projects or emerging misuse patterns, though this is partly mitigated by focusing on long-standing, popular projects.

Internal Validity. Mapping tool-specific rules to consolidated misuse categories involved subjective judgment, which may affect conclusions about tool agreement and comparative performance.

Construct Validity. Our taxonomy reflects the scope of evaluated tools and may not cover all real-world Go misuses, but it is representative of current automated detection capabilities.

6 Related Work

Cryptographic misuse detection has been extensively studied in the Java ecosystem, with several works conducting comparative evaluations of static analysis tools.

Zhang et al. [35] evaluate six tools (CogniCrypt, CryptoGuard, CryptoTutor, FindSecBugs, SonarQube, Xanitizer) across three Java benchmarks and show that developers often reject tool alerts due to low precision. Ami et al. [4] propose MASC, a mutation-based framework for testing detectors under realistic conditions, revealing key practical limitations. Chen et al. [7] analyze false positives and ineffective true positives in CogniCryptSAST, CryptoGuard, and CryptoREX, providing recommendations for improving precision. Wickert et al. [33] study CogniCryptSAST alerts in 210 enterprise projects and find most misuses to be high severity, underscoring their security relevance.

Motivated by recent advances in LLMs, several studies compare them with traditional static analysis for cryptographic misuse detection. Masood and Martin [20] benchmark GPT-4o-mini against CryptoGuard, CogniCrypt, CogniCrypt$_{SAST}$, and Snyk Code in Java. Xia et al. [34] evaluate five LLMs on two Java benchmarks, while Firouzi et al. [12] use prompt engineering with ChatGPT to compare it with CryptoGuard. Overall, LLMs show broader detection coverage but higher false positive rates.

Cryptographic misuse detection has also been explored in Python. Wickert et al. [32] present LICMA, while Frantz et al. [13] introduce PyCryptoBench and evaluate Cryptolation against three industry tools on 940 Python projects. Guo et al. [14] propose CryptoPyt and compare it with five tools across two benchmarks. These works highlight that results from Java do not generalize directly, emphasizing the need for language-specific approaches.

Finally, we review prior work on cryptographic misuse detection in Go. Li et al. introduce CryptoGo, the first Go-specific tool, covering 12 misuse types and uncovering issues across all categories in

120 open-source projects. Building on this, Zhang et al. [36] propose Gopher, improving CryptoGo's detection, and compare both tools to 145 popular Go projects. Their study, however, excludes other Go detectors. We address this gap with the first large-scale comparative evaluation of cryptographic misuse detection tools on real-world, security-relevant Go codebases.

7 Conclusion

This paper presented the first systematic study of cryptographic API misuse detection in Go. We evaluated four tools (CodeQL, Gopher, Gosec, and Snyk Code) across 14 rule classes, reporting 7,473 misuses in 328 open-source projects. Our results reveal major gaps in coverage, precision, and consensus among tools; overlaps are higher-precision, making ensembles a practical near-term strategy while highlighting the need for standardized detection. Future work should combine tool strengths to better support Go security engineers.

Acknowledgments

This work was supported by the Centre for Cyber Defence and Information Security (CDIS), the WASP program funded by Knut and Alice Wallenberg Foundation, and by the Swedish Foundation for Strategic Research (SSF). Some computation was enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS).

References

[1] 2025. *CodeQL for Go (v1.1.13).*
[2] Sharmin Afrose, Ya Xiao, Sazzadur Rahaman, Barton P. Miller, and Danfeng Yao. 2023. Evaluation of Static Vulnerability Detection Tools With Java Cryptographic API Benchmarks. *IEEE Transactions on Software Engineering* 49, 2 (Feb. 2023), 485–497. doi:10.1109/TSE.2022.3154717
[3] Martin R Albrecht, Kenneth G Paterson, and Gaven J Watson. 2009. Plaintext recovery attacks against SSH. In *2009 30th IEEE Symposium on Security and Privacy*. IEEE, 16–26.
[4] Amit Seal Ami, Nathan Cooper, Kaushal Kafle, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. 2022. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. In *2022 IEEE Symposium on Security and Privacy (SP)*. 614–631. doi:10.1109/SP46214.2022.9833582
[5] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. 2016. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Saarbrucken, 292–302. doi:10.1109/EuroSP.2016.31
[6] Jinbao Chen, Boyao Ding, Yu Zhang, Qingwei Li, and Fugen Tang. 2025. An Empirical Study of Cgo Usage in Go Projects: Distribution, Purposes, Patterns and Critical Issues. (February 2025). SSRN:5153961 doi:10.2139/ssrn.5153961
[7] Yikang Chen, Yibo Liu, Ka Lok Wu, Duc V Le, and Sze Yiu Chau. 2024. Towards Precise Reporting of Cryptographic Misuses. In *Proceedings 2024 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2024.241032
[8] CISA. 2014. SSL 3.0 POODLE Attack. https://www.cisa.gov/news-events/alerts/2014/10/17/ssl-30-protocol-vulnerability-and-poodle-attack. Accessed: 2025-05-30.
[9] Cosmin Cojocar, Grant Murphy, and SecureGo Team. 2025. *gosec: Go Security Checker (v2.22.4).* https://github.com/securego/gosec
[10] Russ Cox and Filippo Valsorda. [n. d.]. Secure Randomness in Go 1.22. https://go.dev/blog/chacha8rand. Accessed: 2025-05-30.
[11] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R Crandall. 2015. Analyzing the Great Firewall of China over space and time. *Proceedings on privacy enhancing technologies* (2015).
[12] Ehsan Firouzi, Mohammad Ghafari, and Mike Ebrahimi. 2024. ChatGPT's Potential in Cryptography Misuse Detection: A Comparative Analysis with Static Analysis Tools. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. ACM, Barcelona Spain, 582–588. doi:10.1145/3674805.3695408
[13] Miles Frantz, Ya Xiao, Tanmoy Sarkar Pias, Na Meng, and Danfeng Yao. 2024. Methods and Benchmark for Detecting Cryptographic API Misuses in Python. *IEEE Transactions on Software Engineering* 50, 5 (May 2024), 1118–1129. doi:10.1109/TSE.2024.3377182
[14] Xiangxin Guo, Shijie Jia, Jingqiang Lin, Yuan Ma, Fangyu Zheng, Guangzheng Li, Bowen Xu, Yueqiang Cheng, and Kailiang Ji. 2024. CryptoPyt: Unraveling Python Cryptographic APIs Misuse with Precise Static Taint Analysis. In *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 1075–1091.
[15] Paul E. Hoffman and Bruce Schneier. 2005. Attacks on Cryptographic Hashes in Internet Protocols. RFC 4270. doi:10.17487/RFC4270 Num Pages: 12.
[16] IETF. 2021. Deprecating TLS 1.0 and TLS 1.1. RFC 8996. https://datatracker.ietf.org/doc/rfc8996/
[17] Michael Jones, John Bradley, and Nat Sakimura. 2015. JSON Web Token (JWT). RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519.
[18] Stefan Krüger, Sarah Nadi, Michael Reif, Karim Ali, Mira Mezini, Eric Bodden, Florian Göpfert, Felix Günther, Christian Weinert, Daniel Demmler, et al. 2017. Cognicrypt: Supporting developers in using cryptography. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 931–936.
[19] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. 2022. Cryptogo: Automatic detection of go cryptographic api misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 318–331.
[20] Zohaib Masood and Miguel Vargas Martin. 2024. Beyond Static Tools: Evaluating Large Language Models for Cryptographic Misuse Detection. *arXiv preprint arXiv:2411.09772* (2024).
[21] MITRE. [n. d.]. CWE-1204: Generation of Weak Initialization Vector (IV). https://cwe.mitre.org/data/definitions/1204.html. Accessed: 2025-05-30.
[22] Seyedehzahra Mosavi, Chadni Islam, Muhammad Ali Babar, Sharif Abuadbba, and Kristen Moore. 2023. Detecting Misuse of Security APIs: A Systematic Review. *Comput. Surveys* (2023).
[23] Sarah Nadi, Stefan Krüger, Mira Mezini, and Eric Bodden. 2016. Jumping through hoops: Why do Java developers struggle with cryptography APIs?. In *Proceedings of the 38th International Conference on Software Engineering*. 935–946.
[24] National Institute of Standards and Technology. 2025. NVD - CVE-2025-66491. https://nvd.nist.gov/vuln/detail/CVE-2025-66491. Accessed: 2026-01-25.
[25] Nikhil Patnaik, Joseph Hallett, and Awais Rashid. 2019. Usability Smells: An Analysis of {Developers'} Struggle With Crypto Libraries. In *Fifteenth Symposium on Usable Privacy and Security (SOUPS 2019)*. 245–257.
[26] Luca Piccolboni, Giuseppe Di Guglielmo, Luca P Carloni, and Simha Sethumadhavan. 2021. Crylogger: Detecting crypto misuses dynamically. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1972–1989.
[27] Niels Provos and David Mazieres. 1999. A future-adaptable password scheme.. In *USENIX annual technical conference, FREENIX track*, Vol. 1999. 81–91.
[28] Sazzadur Rahaman, Ya Xiao, Sharmin Afrose, Fahad Shaon, Ke Tian, Miles Frantz, Murat Kantarcioglu, and Danfeng Yao. 2019. Cryptoguard: High precision detection of cryptographic vulnerabilities in massive-sized java projects. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2455–2472.
[29] Snyk. 2025. *Snyk Code for Go (v1.1297.1).* https://docs.snyk.io/supported-languages-package-managers-and-frameworks/go
[30] Meltem Sonmez Turan. 2024. *Keyed-Hash Message Authentication Code (HMAC): Specification of HMAC and Recommendations for Message Authentication.* Technical Report NIST SP 800-224 ipd. National Institute of Standards and Technology, Gaithersburg, MD. NIST SP 800–224 ipd pages. doi:10.6028/NIST.SP.800-224.ipd
[31] The Go Team. 2024. x/crypto/ssh: misuse of ServerConfig.PublicKeyCallback may cause authorization bypass. https://github.com/golang/go/issues/70779. Accessed: 2025-05-30.
[32] Anna-Katharina Wickert, Lars Baumgärtner, Florian Breitfelder, and Mira Mezini. 2021. Python crypto misuses in the wild. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–6.
[33] Anna-Katharina Wickert, Lars Baumgärtner, Michael Schlichtig, Krishna Narasimhan, and Mira Mezini. 2022. To fix or not to fix: a critical study of crypto-misuses in the wild. In *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 315–322.
[34] Yifan Xia, Zichen Xie, Peiyu Liu, Kangjie Lu, Yan Liu, Wenhai Wang, and Shouling Ji. 2024. Exploring Automatic Cryptographic API Misuse Detection in the Era of LLMs. *arXiv preprint arXiv:2407.16576* (2024).
[35] Ying Zhang, Md Mahir Asef Kabir, Ya Xiao, Danfeng Yao, and Na Meng. 2022. Automatic detection of Java cryptographic API misuses: Are we there yet? *IEEE Transactions on Software Engineering* 49, 1 (2022), 288–303.
[36] Yuexi Zhang, Bingyu Li, Jingqiang Lin, Linghui Li, Jiaju Bai, Shijie Jia, and Qianhong Wu. 2024. Gopher: High-Precision and Deep-Dive Detection of Cryptographic API Misuse in the Go Ecosystem. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2978–2992.